



CLAP-ART: Automated Audio Captioning with Semantic-rich Audio Representation Tokenizer

*Daiki Takeuchi, Binh Thien Nguyen, Masahiro Yasuda, Yasunori Ohishi, Daisuke Niizumi,
Noboru Harada*

NTT Corporation, Japan

d.takeuchi@ntt.com

Abstract

Automated Audio Captioning (AAC) aims to describe the semantic contexts of general sounds, including acoustic events and scenes, by leveraging effective acoustic features. To enhance performance, an AAC method, EnCLAP, employed discrete tokens from EnCodec as an effective input for fine-tuning a language model BART. However, EnCodec is designed to reconstruct waveforms rather than capture the semantic contexts of general sounds, which AAC should describe. To address this issue, we propose CLAP-ART, an AAC method that utilizes “semantic-rich and discrete” tokens as input. CLAP-ART computes semantic-rich discrete tokens from pre-trained audio representations through vector quantization. We experimentally confirmed that CLAP-ART outperforms baseline EnCLAP on two AAC benchmarks, indicating that semantic-rich discrete tokens derived from semantically rich AR are beneficial for AAC. **Index Terms:** automated audio captioning, discrete token, residual vector quantization, audio representation.

1. Introduction

Automated Audio Captioning (AAC) is a crossmodal text generation task focused on producing descriptions of diverse sounds [1–3]. Unlike traditional audio classification tasks that assign labels to the sound, AAC seeks to comprehend and articulate the semantic contexts surrounding various acoustic events and scenes.

To enhance the quality of generated captions, some studies have investigated the efficacy of pre-trained language models [4–11]. These approaches involve feeding acoustic features into language models, which enables them to interpret the semantic contexts of the input sounds more effectively.

Recent AAC methodologies employ Audio Representation (AR) models to derive acoustic features that convey semantic information to the language model [6–11]. AR models are trained using a variety of learning strategies. For instance, BEATs [12] employs a combination of self-supervised learning of audio signals along with supervised learning using labels from AudioSet [13]. Similarly, CLAP [14] leverages cross-modal contrastive learning to align audio signals with corresponding text captions. The ability of AR features to capture semantic information has been demonstrated across numerous downstream tasks associated with general sounds. In AAC frameworks, these semantically rich AR features are typically fed into language models in a continuous feature format.

Among the AAC methods that utilize AR features, EnCLAP [6] has shown that performance improves when acoustic features are provided to language models as discrete token representations instead of continuous formats. EnCLAP utilizes EnCodec [15], a neural audio codec, to generate discrete tokens,

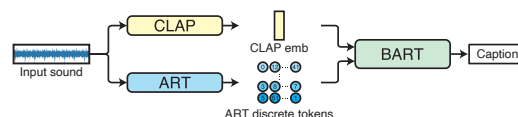


Figure 1: Overview of CLAP-ART. CLAP-ART utilizes ART semantic-rich discrete tokens as BART input along with a CLAP audio embedding.

which are then fed into the BART language model [16] alongside CLAP audio embeddings. This approach has validated the assertion that “pre-trained language models can leverage discrete inputs better compared to their continuous inputs.” This performance enhancement is likely attributable to the fact that standard language models are trained on discrete textual tokens.

However, it should be noted that EnCodec discrete tokens may not be optimal for AAC, as EnCodec is primarily designed to compress audio waveforms into discrete tokens for precise reconstruction of the original waveform rather than producing tokens that encapsulate the semantic richness of the sounds, which AAC aims to describe. It has been found that EnCodec discrete tokens yield inferior performance in various downstream tasks related to general sounds when compared to AR features [17].

Based on the preceding analysis, we hypothesize that “semantic-rich and discrete” token representations can enhance the performance of AAC. These tokens should encapsulate the semantic information intrinsic to general audio, integrating insights from various modalities, such as labels associated with acoustic events and scenes. The acoustic tokenizer utilized in BEATs distills knowledge from BEATs’ audio encoder, enabling it to generate discrete tokens that embody this semantic richness. In contrast, EnCodec primarily focuses on compression and reconstruction, which limits its discrete tokens’ ability to capture semantic richness. Similarly, the SpeechTokenizer [18] produces discrete tokens that derive their information from HuBERT [19], a self-supervised model focused on speech representation. However, because HuBERT’s representation is anchored in phonemes and the linguistic semantics of the spoken language, the discrete tokens generated by the SpeechTokenizer lack the semantic richness necessary for representing general sounds effectively.

In this paper, we propose CLAP-ART, an AAC method that utilizes semantic-rich discrete tokens as input features for a language model. To generate these tokens, we introduce an audio representation tokenizer (ART), which extracts discrete tokens from AR features. We implement two types of ART: (1) BEATs-AT, which naively employs BEATs’ acoustic tokenizer, and (2) BEATs-RVQ, which quantizes BEATs’ audio encoder output by applying k-means clustering in the manner of Residual Vector Quantization (RVQ) [15, 20]. BEATs-AT generates a single-layer discrete token, whereas BEATs-RVQ

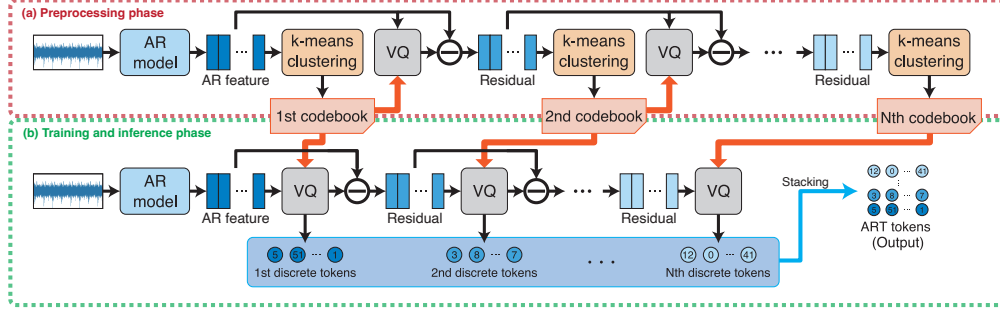


Figure 2: *Details of BEATs-RVQ. The preprocessing phase (a) obtains codebooks in advance of fine-tuning BART. The training and inference phase (b) converts input sound to discrete tokens.*

produces multi-layer discrete tokens, which are expected to enhance AAC performance by providing richer semantic information. CLAP-ART feeds these semantic-rich discrete tokens alongside a CLAP audio embedding into BART to generate captions as illustrated in Fig. 1.

In the experiments to validate our hypothesis, we confirmed that CLAP-ART improves AAC performance. CLAP-ART outperformed the baseline EnCLAP on two AAC benchmarks. We also confirmed that SpeechTokenizer’s discrete tokens did not improve AAC performance, indicating that they are unsuitable for AAC. Furthermore, BEATs-RVQ, which enables multi-layer discrete tokens, outperformed BEATs-AT, verifying that multi-layer discrete tokens can capture more semantic information effectively for AAC. These experiments demonstrate that semantic-rich discrete tokens derived from semantically rich AR are beneficial for AAC.

2. Related work

2.1. Automated Audio Captioning Methods using Pre-trained Language Model

Pre-trained language models have been utilized in some studies to improve AAC performance [4–11]. In order to provide semantic information to language models, pre-trained AR features are used as input. Typical approaches simply pass AR features to language models through a linear layer [7], Conformer layer [8], or Querying Transformer layer [10, 11]. BART-tags [5] provides an AR feature and the BART embedding of the labels predicted by the audio tagging model to the language model BART. RECAP [9] uses captions of similar sounds to an input sound with an AR feature, allowing it to adapt to out-of-domain acoustic events and scenes. Koizumi et al. [4] uses a similar procedure as RECAP to tackle the difficulty of adapting a powerful pre-trained language model to the AAC task.

While AR features are effective for the AAC task, there is still room to consider in the format of acoustic features provided to language models. EnCLAP demonstrated that discrete token representation is more effective for fine-tuning language models than continuous feature representation. The details of EnCLAP are shown in Section 3.

2.2. Audio Discrete Token with Semantic Information

In recent years, several methods have been tried to generate discrete tokens for general sounds. Neural audio codecs, such as EnCodec [15], utilize deep neural networks for compressing an audio waveform into a discrete token representation and reconstructing the original waveform from these discrete tokens. SemantiCodec [17] achieves a lower bitrate codec compared with those of other methods by leveraging pre-trained AR features.

BEATs [12] incorporates an acoustic tokenizer in addition to the audio encoder. This tokenizer outputs single-layer discrete tokens. Since BEATs’ acoustic tokenizer is designed to represent semantic information, it has achieved a level of performance comparable to that of BEATs’ audio encoder in environmental sound classification tasks.

Some studies in speech processing have explored the use of discrete token representations derived from the features of self-supervised speech models and examined their applications [18, 21, 22]. SpeechTokenizer leverages HuBERT’s knowledge to learn unified speech discrete tokens that encapsulate both phonetic and semantic information. The effectiveness of these discrete tokens from SpeechTokenizer has been demonstrated in speech-related tasks such as speech recognition, zero-shot text-to-speech, and one-shot voice conversion. However, the phonetic and semantic information of speech is specific to spoken language and differs from the semantic context of general sounds, which is the focus of AAC. We used SpeechTokenizer as a comparison method to validate the difference between speech semantics and general sound semantics.

3. Baseline: EnCLAP

EnCLAP leverages EnCodec alongside the CLAP audio encoder to extract input features for BART fine-tuning. First, EnCLAP transforms an input audio waveform into a CLAP audio embedding and EnCodec discrete tokens. The CLAP audio embedding is projected onto the BART input embedding space by using a linear layer, resulting in $e_{\text{clap}} \in \mathbb{R}^{D_b}$, where D_b is the hidden size of BART. EnCodec discrete tokens are converted into embeddings by using embedding layers corresponding to the respective layers of discrete tokens and then summed to form a sequence $e_{\text{encodec}} \in \mathbb{R}^{L \times D_b}$, where L is the time length of the discrete tokens. Next, e_{encodec} is enclosed with BART’s special tokens $\langle \text{bos} \rangle$ and $\langle \text{eos} \rangle$. And then, positional embeddings e_{pos} is applied to it:

$$I_{\text{seq}} = [e_{\text{bos}}, e_{\text{encodec}}, e_{\text{eos}}] + e_{\text{pos}} \in \mathbb{R}^{(L+2) \times D_b}. \quad (1)$$

Finally, e_{clap} and I_{seq} are concatenated to produce input for BART:

$$I_{\text{EnCLAP}} = [e_{\text{clap}}, I_{\text{seq}}] \in \mathbb{R}^{(L+3) \times D_b}. \quad (2)$$

EnCLAP is trained using a combination of the cross-entropy loss $\mathcal{L}_{\text{caption}}$ for caption generation and the masked codec modeling (MCM) loss $\mathcal{L}_{\text{mcm}}$. The total loss is given by $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{caption}} + \lambda \mathcal{L}_{\text{mcm}}$, where λ is a weighting parameter.

4. Proposed Method

CLAP-ART leverages the audio representation tokenizer (ART) proposed in this paper alongside the CLAP audio encoder to extract input features for BART fine-tuning.

4.1. Audio Representation Tokenizer

ART converts input sounds into semantic-rich discrete tokens based on the knowledge of AR models. We implemented two types of ART: BEATs-AT and BEATs-RVQ. BEATs-AT naively uses BEATs’ acoustic tokenizer. BEATs-RVQ is a newly designed ART, which quantizes continuous features output from BEATs’ audio encoder by RVQ [15, 20].

The details of BEATs-RVQ are illustrated in Fig. 2. Prior to the training, we create the RVQ codebooks on the training dataset by using k-means clustering of the continuous audio features of audio samples encoded by the AR model. Although the BEATs’ audio encoder outputs $768 \times FT$ features from $F \times T$ (frequency $\times$ time) patches, BEATs-RVQ feeds them directly into RVQ as a sequence of 768-dimensional vectors (length FT) without reshaping them to reflect the original time-frequency structure (e.g., $768F \times T$). In the training and inference stage, the AR model encodes the input into features, and then the RVQ layer vector-quantizes the features into discrete tokens. The VQ procedure follows the original RVQ steps in [15, 20]: Project the input vector (feature) onto the closest entry in the first codebook, compute the residual after quantization, quantize it using the next codebook, and repeat the last two steps until the last codebook. BEATs-RVQ can generate multi-layer discrete tokens, which are expected to improve AAC performance by packing more semantic information than in BEATs-AT discrete tokens, which are only single-layer.

4.2. Training with ART Discrete Tokens

We trained CLAP-ART on the EnCLAP framework. CLAP-ART calculates embeddings $e_{\text{art}} \in \mathbb{R}^{L \times D_b}$ from ART discrete tokens by using the same procedure for embedding EnCodec discrete tokens. e_{art} is used instead of e_{encodec} as follows:

$$I_{\text{seq}} = [e_{\text{bos}}, e_{\text{art}}, e_{\text{eos}}] + e_{\text{pos}} \in \mathbb{R}^{(L+2) \times D_b}, \quad (3)$$

$$I_{\text{CLAP-ART}} = [e_{\text{clap}}, I_{\text{seq}}] \in \mathbb{R}^{(L+3) \times D_b}. \quad (4)$$

CLAP-ART uses $I_{\text{CLAP-ART}}$ instead of I_{EnCLAP} and follows the EnCLAP’s training procedure as mentioned in Section 3.

5. Experiments

5.1. Experimental Setup

We conducted experiments on two AAC datasets: AudioCaps [23] and Clotho [24]. For the experiments on AudioCaps, we trained and evaluated models solely on AudioCaps. For the experiments on Clotho, we first pre-trained models on AudioCaps and then fine-tuned and evaluated them on Clotho.

In the setup of CLAP-ART, we utilized the acoustic tokenizer and the audio encoder of BEATs_{iter3+} for BEATs-AT and BEATs-RVQ, respectively. For BEATs-RVQ, the number of RVQ layers and codebook size were 16 and 1024, respectively, which were the same as those for EnCodec in EnCLAP. We used BART as the pre-trained language model to generate captions. Basically, the base model of BART was used. The experiments shown in Table 4 used both the base and large models of BART.

We used the same training and inference settings with EnCLAP for the sake of simple comparison, except for the learn-

ing rate and pre-training setup for Clotho, which we set based on grid search. The number of epochs was 15. The loss weighting parameter λ was fixed to 0.7. 15 % of the input tokens were masked for MCM. The mask length was 10. The optimizer was AdamW [25] with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. Weight decay was set to 0.01. Label smoothing where a factor was 0.2 was used for loss for only text generation $\mathcal{L}_{\text{caption}}$. We used the inverse square root learning rate scheduler with a linear warm-up. The experiment on AudioCaps used a peak learning rate of 6e-5 and 2000 warm-up steps. In the experiment on Clotho, we conducted pre-training on AudioCaps in 5 epochs and then trained on Clotho with a peak learning rate of 2e-4 and no warmup. The inference used a beam search with a beam size of 4. We implemented the above in accordance with the official implementation of EnCLAP¹ to reproduce the baseline.

To ensure that the results of this research are reproducible, we specifically experimented with EnCLAP, CLAP-ART, and other variations using six different random seeds and calculated statistics as the final results. For the evaluation metrics, we computed METEOR [26], CIDEr [27], SPICE [28], SPIDER [29], and FENSE [30] by using the *aac-metrics* library².

5.2. Comparison with baseline and our CLAP-ART

We found that CLAP-ART, supported by ART discrete tokens, improves AAC performance by comparing it with EnCLAP. The results are shown in Table 1. EnCLAP is the baseline of this experiment. CLAP-SpeechTokenizer used SpeechTokenizer’s discrete tokens instead of ART ones. CLAP-HuBERT-RVQ converted HuBERT features into discrete tokens by using the same procedure as BEATs-RVQ. CLAP-ART(BEATs-AT) and CLAP-ART(BEATs-RVQ) utilized BEATs-AT and BEATs-RVQ as ART, respectively. Each score denotes the average score and sample standard deviation of six experiments with different random seeds.

From these results, we confirmed that CLAP-ART outperforms EnCLAP across all evaluation metrics and benchmarks, supporting our hypothesis that semantic-rich discrete tokens are beneficial to enhancing AAC performance. In particular, both CLAP-ART(BEATs-AT) and CLAP-ART(BEATs-RVQ) demonstrated more significant improvements on AudioCaps. For instance, CLAP-ART(BEATs-RVQ) improved the SPIDER score to +3.5, while that on Clotho was +1.3. Since the BEATs_{iter3+} was pre-trained using the AudioSet supervision, it is reasonable for it to be effective for AudioCaps, a subset of AudioSet. CLAP-SpeechTokenizer and CLAP-HuBERT-RVQ did not improve AAC performance from the level of EnCLAP, indicating that knowledge of speech phonemes and linguistic semantics is not beneficial for AAC.

5.3. Ablation study of CLAP audio embedding

To validate the semantic richness of discrete tokens generated by ART, we conducted an ablation experiment on the CLAP audio embedding on AudioCaps. We compared the performances of EnCLAP and CLAP-ART(BEATs-RVQ) with and without the CLAP embedding input enabled. When disabled, the AAC system relied solely on the semantic information in the discrete tokens of the EnCodec for EnCLAP or those of BEATs-RVQ for CLAP-ART.

Table 2 shows the average scores and the difference between with and without the CLAP embedding. The columns of “Diff” clearly show that degradation in CLAP-ART(BEATs-

¹<https://github.com/jaeyeonkim99/EnCLAP>

²<https://github.com/Labbeti/aac-metrics>

Table 1: Comparison of CLAP-ART with conventional methods on AudioCaps and Clotho. Bold indicates the best score for each metric.

Method	AudioCaps					Clotho (Pre-trained on AudioCaps)				
	METEOR	CIDEr	SPICE	SPIDEr	FENSE	METEOR	CIDEr	SPICE	SPIDEr	FENSE
Conventional										
EnCLAP [6] (CLAP-EnCodec)	24.2 $\pm$ 0.36	74.4 $\pm$ 1.43	18.1 $\pm$ 0.21	46.3 $\pm$ 0.76	64.2 $\pm$ 0.47	18.2 $\pm$ 0.11	45.4 $\pm$ 0.69	12.8 $\pm$ 0.20	29.1 $\pm$ 0.38	50.6 $\pm$ 0.19
CLAP-SpeechTokenizer	24.3 $\pm$ 0.08	74.7 $\pm$ 1.15	17.8 $\pm$ 0.45	46.3 $\pm$ 0.76	64.2 $\pm$ 0.41	18.2 $\pm$ 0.15	45.3 $\pm$ 0.70	12.9 $\pm$ 0.15	29.1 $\pm$ 0.42	50.1 $\pm$ 0.28
CLAP-HuBERT-RVQ	24.5 $\pm$ 0.42	74.6 $\pm$ 1.59	18.0 $\pm$ 0.16	46.3 $\pm$ 0.83	64.6 $\pm$ 0.33	18.2 $\pm$ 0.15	45.8 $\pm$ 0.72	13.1 $\pm$ 0.13	29.4 $\pm$ 0.39	50.4 $\pm$ 0.28
Proposed										
CLAP-ART(BEATs-AT)	25.1 $\pm$ 0.45	78.0 $\pm$ 1.30	18.4 $\pm$ 0.25	48.2 $\pm$ 0.74	64.9 $\pm$ 0.32	18.6 $\pm$ 0.11	47.3 $\pm$ 0.48	13.2 $\pm$ 0.10	30.3 $\pm$ 0.20	50.8 $\pm$ 0.13
CLAP-ART(BEATs-RVQ)	25.6 $\pm$ 0.39	80.7 $\pm$ 1.22	18.8 $\pm$ 0.25	49.8 $\pm$ 0.61	65.5 $\pm$ 0.11	18.7 $\pm$ 0.16	47.5 $\pm$ 1.16	13.3 $\pm$ 0.16	30.4 $\pm$ 0.65	51.1 $\pm$ 0.55

Table 2: Ablation study of CLAP audio embedding on AudioCaps to verify semantic-richness of ART discrete tokens.

Method	METEOR		CIDEr		SPICE		SPIDEr		FENSE	
	Mean	Diff	Mean	Diff	Mean	Diff	Mean	Diff	Mean	Diff
EnCLAP-base [6]	24.2	-	74.4	-	18.2	-	46.3	-	64.2	-
w/o CLAP ^b	18.9	-5.3	48.4	-26.0	13.0	-5.2	30.7	-15.6	49.7	-14.5
CLAP-ART(BEATs-RVQ)	25.6	-	80.7	-	18.8	-	49.8	-	65.5	-
w/o CLAP ^b	25.0	-0.6	77.1	-3.6	18.1	-0.7	47.6	-2.2	63.9	-1.6

^b CLAP audio embedding is disabled. That is, I_{enc} is used for I_{encCLAP} or $I_{\text{CLAP-ART}}$.

Table 3: Comparison with variation of the number of RVQ layers in BEATs-RVQ on AudioCaps. N indicates the number of RVQ layers.

Method	N	METEOR	CIDEr	SPICE	SPIDEr	FENSE
CLAP-ART(BEATs-RVQ)	16	25.6	80.7	18.8	49.8	65.5
	8	25.5	80.6	18.8	49.7	65.2
	4	25.3	78.6	18.6	48.6	65.0
	2	25.2	79.2	18.4	48.8	65.1
	1	25.0	78.6	18.4	48.5	65.0
CLAP-ART(BEATs-AT)	1	25.1	78.0	18.4	48.2	64.9

Table 4: Comparison with SOTA methods on AudioCaps. #LM Params. indicates the number of parameters of the pre-trained language model.

Method	#LM Params.	METEOR	CIDEr	SPICE	SPIDEr	FENSE
Reported in their papers						
BART-tags [5]	406M	24.1	75.3	17.6	46.5	-
HTSAT-BART ⁺ [7]	139M	25.0	78.7	18.2	48.5	-
AutoCap (w/o metadata input) [10]	139M	24.6	77.3	18.2	47.8	-
AutoCap (w/o metadata input) ⁺ [10]	139M	25.6	80.4	19.0	49.7	-
EnCLAP-base [*] [6]	139M	24.7	78.0	18.6	48.3	-
EnCLAP-large [*] [6]	406M	25.5	80.3	18.8	49.5	-
LOAE ⁺ [11]	7B	26.7	81.6	19.3	50.5	66.2
Ours (Averaged scores)						
CLAP-ART(BEATs-RVQ)-base ⁺	139M	25.6	80.7	18.8	49.8	65.5
CLAP-ART(BEATs-RVQ)-large ⁺	406M	25.7	80.8	19.1	50.0	65.4
The best performing system results of ours (Reference)						
CLAP-ART(BEATs-RVQ)-base ⁺	139M	25.8	82.0	19.0	50.5	65.4
CLAP-ART(BEATs-RVQ)-large ⁺	406M	25.9	81.5	19.6	50.6	65.2

⁺ Methods used pre-training with extra audio-text paired datasets such as WavCaps.

^{*} Only EnCLAP and CLAP-ART utilized discrete tokens as input features.

[†] Average scores in six experiments.

[‡] The scores obtained by the system with the best spider score in six experiments.

RVQ)’s scores was smaller than the EnCLAP’s (compare, e.g., CLAP-ART(BEATs-RVQ)’s -2.2 vs. EnCLAP’s -15.6 in SPIDEr). CLAP-ART(BEATs-RVQ) w/o CLAP also achieved comparable scores to EnCLAP using CLAP (47.6 vs. 46.3 in SPIDEr). These results demonstrated that ART discrete tokens effectively capture semantic information of general sounds, improving AAC performance.

5.4. Verification that semantic-rich multi-layer discrete tokens from BEATs-RVQ capture more semantic information

This experiment investigated whether BEATs-RVQ improves AAC performance by capturing more semantic information through multi-layer codebooks. In addition to the results shown in Table 1, we further evaluated several variants of CLAP-ART (BEATs-RVQ) with different numbers of RVQ layers.

The results are shown in Table 3. CLAP-ART (BEATs-AT) and CLAP-ART (BEATs-RVQ) with $N = 1$ showed mostly equivalent performance. CLAP-ART (BEATs-RVQ) can leverage RVQ to increase the number of layers to $N = 16$, achieving higher scores than CLAP-ART (BEATs-AT). This demon-

strates that multi-layer discrete tokens from BEATs-RVQ improve AAC performance by packing more semantic information.

5.5. Comparison with state-of-the-art AAC methods

We conducted a comparison with conventional AAC methods. As shown in Table 4, the results from conventional methods are based on their original papers. Among these, HTSAT-BART [7] and LOAE [11] were trained with extra audio-text paired datasets such as WavCaps [7] (denoted by ⁺), whereas BART-tags [5], EnCLAP [6], and our CLAP-ART were trained using only AudioCaps. AutoCap [10] reported the results for both versions: one trained solely on AudioCaps and another utilizing extra datasets. In this experiment, we used only audio input for both versions of AutoCap to compare with other methods. Although we focused on audio-only input, AutoCap has the potential to improve caption quality by incorporating textual metadata as an additional input. As for our CLAP-ART(BEATs-RVQ), we compared two systems: one using BART’s base model (CLAP-ART(BEATs-RVQ)-base) and the other using BART’s large model (CLAP-ART(BEATs-RVQ)-large). For both CLAP-ART(BEATs-RVQ), we also report the metric scores of the model that achieved the best SPIDEr score out of six experiments as a reference.

The results showed that even with the base model of BART, CLAP-ART(BEATs-RVQ)-base outperformed all other methods, except for LOAE [11]. The metric scores of CLAP-ART(BEATs-RVQ)-base were averaged over six experiments with different random seeds, demonstrating the superior performance of CLAP-ART(BEATs-RVQ)-base regardless of random seed variations. CLAP-ART(BEATs-RVQ)-large further improved the averaged metric scores. The best-performing CLAP-ART(BEATs-RVQ) models among the six experiments achieved a SPIDEr score comparable to that of LOAE, an LLM-based method. These results suggest that the performance improvement based on our hypothesis in CLAP-ART is a significant contribution comparable to the use of LLM.

6. Conclusions

This paper proposed CLAP-ART, which improves AAC performance by utilizing “semantic-rich and discrete” tokens, which capture fine-grained semantic contexts of general sounds. CLAP-ART is based on our hypothesis that semantic-rich discrete tokens are beneficial inputs for fine-tuning language models. To obtain such discrete tokens, we designed ART by leveraging semantically rich AR. We implemented CLAP-ART to take ART discrete tokens and CLAP audio embedding as input for BART. Experiments on two AAC benchmarks showed that CLAP-ART outperformed the baseline EnCLAP, supporting our hypothesis.

Future work includes investigating the integration of LLM-based methodologies and post-processing strategies, such as reranking [31] and CLAP-based text decoding [32].

7. Acknowledgements

This work was partially supported by JST Strategic International Collaborative Research Program (SICORP), Grant Number JPMJSC2306, Japan.

8. References

- [1] K. Drossos, S. Adavanne, and T. Virtanen, "Automated audio captioning with recurrent neural networks," in *Workshop Appl. Signal Process. Audio Acoust. (WASPAA)*. IEEE, 2017, pp. 374–378.
- [2] X. Mei, X. Liu, M. D Plumbley, and W. Wang, "Automated audio captioning: An overview of recent progress and new challenges," *EURASIP J. Audio Speech Music Process.*, vol. 2022, no. 1, pp. 26, 2022.
- [3] X. Xu, Z. Xie, M. Wu, and K. Yu, "Beyond the status quo: A contemporary survey of advances and challenges in audio captioning," *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 32, pp. 95–112, 2024.
- [4] Y. Koizumi, Y. Ohishi, D. Niizumi, D. Takeuchi, and Masahiro Y., "Audio captioning using pre-trained large-scale language model guided by audio-based similar caption retrieval," *arXiv preprint arXiv:2012.07331*, 2020.
- [5] F. Gontier, R. Serizel, and C. Cerisara, "Automated audio captioning by fine-tuning bart with audioset tags," in *Proc. Workshop Detect. Classif. Acoust. Scenes Events (DCASE)*, 2021.
- [6] J. Kim, J. Jung, J. Lee, and S. H. Woo, "Enclap: Combining neural audio codec and audio-text joint embedding for automated audio captioning," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2024, pp. 6735–6739.
- [7] X. Mei, C. Meng, H. Liu, Q. Kong, T. Ko, C. Zhao, M. D Plumbley, Y. Zou, and W. Wang, "Wavcaps: A chatgpt-assisted weakly-labelled audio captioning dataset for audio-language multimodal research," *IEEE/ACM Trans. Audio Speech Lang. Process.*, 2024.
- [8] S.-L. Wu, X. Chang, G. Wichern, J. Jung, F. Germain, J. Le Roux, and S. Watanabe, "Improving audio captioning models with fine-grained audio features, text embedding supervision, and llm mix-up augmentation," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*. IEEE, 2024, pp. 316–320.
- [9] S. Ghosh, S. Kumar, C. K. Reddy Evuru, R. Duraiswami, and D. Manocha, "Recap: Retrieval-augmented audio captioning," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2024, pp. 1161–1165.
- [10] M. Haji-Ali, W. Menapace, A. Siarohin, G. Balakrishnan, S. Tulyakov, and V. Ordenez, "Taming data and transformers for audio generation," *arXiv preprint arXiv:2406.19388*, 2024.
- [11] J. Liu, G. Li, J. Zhang, H. Dinkel, Y. Wang, Z. Yan, Y. Wang, and B. Wang, "Enhancing automated audio captioning via large language models with optimized audio encoding," in *Proc. Interspeech*, 2024.
- [12] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, W. Che, X. Yu, and F. Wei, "BEATs: Audio pre-training with acoustic tokenizers," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Jul. 2023, pp. 5178–5193.
- [13] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2017, pp. 776–780.
- [14] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [15] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *Trans. Mach. Learn. Res.*, 2023.
- [16] Mike Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annu. Meet. Assoc. Comput. Linguist. (ACL)*, 2020, pp. 7871–7880.
- [17] H. Liu, X. Xu, Y. Yuan, M. Wu, W. Wang, and M. D Plumbley, "Semanticcodec: An ultra low bitrate semantic audio codec for general sound," *arXiv preprint arXiv:2405.00233*, 2024.
- [18] X. Zhang, D. Zhang, S. Li, Y. Zhou, and X. Qiu, "Speechtokenizer: Unified speech tokenizer for speech language models," in *Proc. Int. Conf. Learn. Represent.*, 2024.
- [19] W.-N. Hsu, B. Bolte, Y.-H H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 29, pp. 3451–3460, 2021.
- [20] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi, "SoundStream: An end-to-end neural audio codec," *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 30, pp. 495–507, 2022.
- [21] Alexei Baevski, Steffen Schneider, and Michael Auli, "vq-wav2vec: Self-supervised learning of discrete speech representations," in *International Conference on Learning Representations*, 2020.
- [22] P. Mousavi, J. Duret, S. Zaiem, L. Della Libera, A. Ploujnikov, C. Subakan, and M. Ravanelli, "How should we extract discrete audio tokens from self-supervised models?," in *Interspeech 2024*, 2024, pp. 2554–2558.
- [23] C. D. Kim, B. Kim, H. Lee, and G. Kim, "Audiocaps: Generating captions for audios in the wild," in *Proc. Conf. N. Am. Chapter Assoc. Comput. Linguist. (NAACL)*, 2019, pp. 119–132.
- [24] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An audio captioning dataset," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*. IEEE, 2020, pp. 736–740.
- [25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.
- [26] M. Denkowski and A. Lavie, "Meteor universal: Language specific translation evaluation for any target language," in *Proc. Workshop Stat. Mach. Transl.*, June 2014, pp. 376–380.
- [27] R. Vedantam, C. Lawrence Zitnick, and D. Parikh, "CIDEr: Consensus-based image description evaluation," in *IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 4566–4575.
- [28] P. Anderson, B. Fernando, M. Johnson, and S. Gould, "SPICE: Semantic propositional image caption evaluation," in *Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 382–398.
- [29] S. Liu, Z. Zhu, N. Ye, S. Guadarrama, and K. Murphy, "Improved image captioning via policy gradient optimization of spider," in *IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 873–881.
- [30] Z. Zhou, Z. Zhang, X. Xu, Z. Xie, M. Wu, and K. Q. Zhu, "Can audio captions be evaluated with image caption metrics?," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*. IEEE, 2022, pp. 981–985.
- [31] J. Kim, M. Jeon, J. Jung, S. H. Woo, and J. Lee, "Enclap++: Analyzing the enclap framework for optimizing automated audio captioning performance," in *Proc. Workshop Detect. Classif. Acoust. Scenes Events (DCASE)*, 2024, pp. 61–65.
- [32] W. Chen, Z. Ma, X. Li, X. Xu, Y. Liang, Z. Zheng, K. Yu, and X. Chen, "Slam-aac: Enhancing audio captioning with paraphrasing augmentation and clap-refine through llms," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2025.